

NightWatch – An Intelligent EDR Framework

*A Project Report submitted in partial fulfilment of the requirements
for the award of the degree of*

Bachelor of Technology

in

Computer Science and Engineering

by

Name of Students

Roll No:

Nitin Sikarwar (2315300014)

Piyush Kumar (2315300015)

Sumit Swain (2315300031)

Under the Guidance of

Mr. Samsher Khan,

Department of Computer Engineering & Applications

Institute of Engineering & Technology



GLA University

Mathura- 281406, INDIA

May, 2019



Department of Computer Engineering and Applications
GLA University, 17 km. Stone NH#2, Mathura-Delhi Road,
Chaumuha, Mathura – 281406 U.P (India)

Declaration

I hereby declare that the work which is being presented in the B.Tech. Project “**NightWatch – An Intelligent EDR Framework**”, in partial fulfillment of the requirements for the award of the *Bachelor of Technology* in Computer Science and Engineering and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Mr. Samsher Khan**.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Name of Candidate:
University Roll No.:

Sign _____

Name of Candidate:
University Roll No.:

Sign _____

Name of Candidate:
University Roll No.:

Sign _____

Name of Candidate:
University Roll No.:

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor

Mr./Ms./Mrs XYZ

Designation of Supervisor

Project Coordinator

(Mr. Mayank Srivastava)

Program Co-ordinator

(Dr Anant Ram)

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty guide, **Mr. Samsher Khan**, for their continuous support, guidance, and valuable feedback throughout the duration of this project. I also thank the **Department of CEA** for providing the necessary resources and environment to complete this work. Lastly, I appreciate the encouragement and cooperation of my friends and family, without whom this project would not have been possible.

⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮
⋮

Sign _____

Name of Candidate: Nitin Sikarwar

University Roll No: 2315300014

Sign _____

Name of Candidate: Piyush Kumar

University Roll No: 2315300015

Sign _____

Name of Candidate: Sumit Kumar Swain

University Roll No: 2315300031

ABSTRACT

NightWatch is an intelligent Endpoint Detection and Response (EDR) framework designed to provide real-time visibility, threat detection, and automated response capabilities across enterprise endpoints. With the increasing sophistication of cyber threats, traditional antivirus solutions and signature-based tools fail to adequately protect modern systems. NightWatch addresses this gap by integrating lightweight endpoint agents, centralized analytics, and machine-learning-driven threat evaluation into a unified security platform.

The endpoint agent continuously collects telemetry such as process activity, file system changes, network connections, and behavioral anomalies. This data is securely transmitted to a central server where it is analyzed using heuristic rules, anomaly detection algorithms, and correlation logic to identify malicious patterns. The system prioritizes minimal performance overhead, ensuring real-time monitoring without disrupting endpoint operations.

NightWatch features a modular architecture that includes telemetry ingestion APIs, a scalable database layer, and a web-based dashboard for incident visualization and management. Security teams can monitor live activity, investigate suspicious behavior, and initiate automated or manual responses such as process termination, file quarantine, or network isolation.

The intelligent response engine enhances detection accuracy through adaptive learning, enabling NightWatch to evolve with new attack vectors and reduce false positives. Additionally, the dashboard provides intuitive insights, alert prioritization, and endpoint health metrics to support faster decision-making.

Overall, NightWatch serves as a comprehensive and efficient EDR framework tailored for academic research and practical deployment scenarios. Its combination of intelligent detection, lightweight design, and user-friendly interface makes it a robust solution for strengthening endpoint security in modern threat landscapes.

CONTENTS

Declaration	ii
Certificate	ii
Acknowledge	iii
Abstract	iv
List of figures	v
List of Tables	vi
CHAPTER 1 Introduction	1
1.1 Overview and Motivation	1
1.2 Objective	1
1.3 Summary of Similar Application	2
CHAPTER 2 Software Requirement Analysis	3
2.1 Technical Feasibility	4
2.2 Hardware and Software Requirements	5
CHAPTER 3 Software Design	6
3.1 System Design Overview	6
3.2 Architecture and Module Description	6
CHAPTER 4 Implementation and User Interface	8
CHAPTER 5 Software Testing	12
CHAPTER 6 Conclusion	14
CHAPTER 7 Summary	14

Chapter 1

Introduction

Cybersecurity has become an essential requirement for every modern organization as attackers continuously search for weak points in systems and networks. Endpoints such as laptops, desktops and servers are one of the most common targets because they directly interact with users, external devices and the internet. If an endpoint is compromised, attackers can steal data, spread malware and move laterally across the environment.

1.1 Motivation and Overview

Traditional antivirus solutions mainly rely on signature-based detection, where known malware patterns are matched against files or processes. While this method is effective for already known threats, it often fails against new, unknown or fileless attacks. To address these limitations, security teams have shifted towards Endpoint Detection and Response (EDR) solutions that provide continuous monitoring, behaviour-based detection and detailed telemetry from endpoints. This project, titled “NightWatch – An Intelligent EDR Framework”, focuses on building a mini-EDR system using Python that is suitable for academic use, security research and small organizations.

NightWatch demonstrates how endpoint telemetry can be collected from the operating system, how heuristics can be applied to this telemetry to identify suspicious activity and how alerts can be visualized in a dashboard. The goal is not to replace commercial EDR tools but to give students and security enthusiasts a hands-on framework that explains EDR concepts in a clear and practical way.

1.2 Motivation of Project

The motivation behind this project arises from the rapidly evolving cybersecurity landscape, where traditional antivirus solutions are no longer sufficient to detect modern threats. Attackers increasingly use techniques such as **fileless malware**, **lateral movement**, and **living-off-the-land binaries (LOLBins)**, which often go undetected by signature-based tools. Organizations, students and small institutions need a **lightweight, transparent and educational EDR-like solution** that helps them understand how endpoint monitoring, behavioural analysis and alert generation truly work. This project aims to fill that gap by creating a Python-based EDR framework that can actively monitor processes, file system activity, network behaviour and suspicious system patterns. The motivation also stems from the need for a system that is **easy to**

extend, modular, and suitable for academic learning environments where hands-on understanding is more important than enterprise-level complexity.

Additionally, the project is motivated by the demand for a tool that can demonstrate **real EDR concepts** without requiring expensive licenses or complex deployments.

NightWatch allows learners to practically explore how detection happens using:

- **Behaviour-based rules** (e.g., risky directory execution, encoded PowerShell)
- **System telemetry collection** (process, network, file changes)
- **Alert categorization and scoring**
- **Real-time visualization via a dashboard**

This combination of practicality, accessibility and educational value forms the core motivation behind developing NightWatch as an intelligent EDR framework.

1.3 Summary of Similar Application

Before developing **NightWatch**, it is important to understand existing tools and how they differ in purpose and design. Several categories of tools are relevant:

Traditional Antivirus Software - Mainly uses signature-based detection with some heuristic or behavioural features. - Effective against known malware but weaker against zero-day or fileless attacks. - Provides limited visibility into process chains, registry persistence, detailed network behaviour and correlated signals.

Commercial EDR Solutions - Examples include **CrowdStrike Falcon**, **SentinelOne** and **Microsoft Defender** for Endpoint. They provide deep visibility, cloud-based analytics and rich threat intelligence. They are proprietary and expensive, often not suitable for students or small organizations. Their internal working is usually hidden, making them unsuitable as learning platforms.

Open-Source Host Monitoring Tools such as OSSEC, Wazuh and OSQuery offer host-based intrusion detection and query-based telemetry. They are powerful but relatively complex for beginners and often require a separate server and analytics layer. - They may not provide an integrated, minimal example suited for teaching EDR concepts step by step.

NightWatch is different because it is explicitly designed as a small, educational, Python-based EDR-style framework. It focuses on clarity and modularity rather than enterprise completeness. It includes:

Behaviour-based heuristics such as suspicious parent-child processes, risky directory execution and encoded PowerShell commands. Hash-based checks of executables using SHA-256 and a simple known-bad list. A built-in Flask dashboard, so no external SIEM or large system is needed.

CHAPTER 2 SOFTWARE REQUIREMENT ANALYSIS

Any software project must begin with a careful study of requirements to ensure that the final system is feasible, practical and aligned with user expectations. For NightWatch, requirement analysis covers technical feasibility, hardware and software needs and basic operational considerations. Understanding these requirements helps avoid unrealistic design choices and ensures smooth implementation.

This chapter describes the technical feasibility of building a Python-based EDR framework, lists the minimum hardware and software components required and highlights why this project is suitable for an academic and lab environment.

2.1 Technical Feasibility

Technical feasibility evaluates whether the technology, tools and skills required for the project are available and appropriate. NightWatch is designed to be technically feasible using widely available, open-source technologies.

Key technical elements include:

Programming Language: Python 3.x is used for both the monitoring engine and the web dashboard. It is easy to learn, cross-platform and has rich libraries for system programming.

Libraries and Frameworks (Agent Side):

- psutil: for reading process details, CPU usage, memory usage and network connections.
- watchdog or directory polling: for monitoring file system changes in real time.
hashlib: for computing SHA-256 hashes of files.
- OS, time, datetime, collections: core Python modules for operating system interaction, timing and data structures.
- winreg and Windows event log APIs (optional): for reading registry Run/RunOnce keys and selected Windows event logs.
- Libraries and Frameworks (Dashboard Side):
 - Flask: a lightweight Python web framework used to build REST APIs and render HTML templates.
 - HTML, CSS and JavaScript: for creating the user interface in a web browser.

- Operating System Support: - Endpoint agent is optimized for Windows 10/11 to take advantage of the registry and event logs. - The Flask dashboard can run on Windows or Linux, as long as Python is available.

From a technical point of view, the project is feasible because it avoids kernel-level drivers and uses user-space APIs that are stable and well documented. All required libraries are open-source and can be installed using standard Python package managers.

2.2 Hardware and Software Requirements

For successful deployment and testing of NightWatch in a lab or college environment, the hardware and software requirements are modest. This makes the system accessible for students and small organizations.

Minimum hardware requirements include:

- CPU: Dual-core processor or better. - RAM: At least 4 GB of memory to run the operating system, agent and dashboard comfortably.
- Storage: At least 10 GB of free disk space to store logs, temporary files and supporting software.
- Network: Basic local network or internet connectivity to access the dashboard from other machines, if needed.

Software requirements for the endpoint (agent machine) are:

- Operating System: Windows 10 or Windows 11 is recommended for full feature support.
- Software: Python 3.x installed on the system. Required Python libraries installed using pip (such as psutil, watchdog and Flask if the dashboard is also on this machine).

Software requirements for the dashboard (server machine) are:

- Operating System: Windows or any modern Linux distribution.
- Software: Python 3.x and Flask for running the web server. A modern web browser (such as Google Chrome, Mozilla Firefox or Microsoft Edge) to view the dashboard.

Additional considerations are:

For small demonstrations, the agent and dashboard can run on the same machine. - For multi-endpoint setups, the dashboard can be hosted on a central server while multiple agents send telemetry over the network.

These requirements show that NightWatch can be deployed without specialized hardware, making it economically and operationally feasible.

CHAPTER 3

SOFTWARE DESIGN

Good software design ensures that a project is organized, maintainable and extendable. For an EDR-style system like NightWatch, design decisions directly affect performance, clarity and security. Software design covers the definition of modules, data flow between components and the separation of responsibilities so that each part of the system has a clear role.

This chapter explains the overall architectural design of NightWatch, describes the main modules and shows how data flows from the monitoring engine to the dashboard. It also discusses how the detection logic and alerting are integrated into the design.

3.1 System Design Overview

NightWatch uses a modular architecture in which the agent, detection logic, alert management and dashboard work together but remain clearly separated. This makes the system easier to read, debug and extend.

At a high level, the system works as follows:

- The monitoring engine (agent) continuously collects telemetry about processes, network connections, file system changes and optional registry or event logs.
- The detection and scoring logic analyse this telemetry, calculate scores for suspicious behaviour and generate alerts when thresholds are crossed.
- The alert management module stores alerts in a bounded queue and maintains counters by severity and category.
- The Flask-based web dashboard exposes APIs to retrieve summaries, alerts, process lists, network data and statistics.
- The user accesses the dashboard through a web browser to monitor the endpoint in real time.
- This separation of concerns ensures that changes in one area (such as rules in the detection logic) do not require large changes in other areas (such as UI rendering).

3.2 Architecture and Module Description

The architecture of NightWatch can be broken into the following modules:

Monitoring Modules (Agent Side)

Process Monitor: Uses psutil to enumerate processes, read attributes such as PID, name, executable path, command line, CPU and memory usage. Applies heuristics to identify suspicious processes, such as execution from risky directories, unusual parent-child chains, long or encoded command lines and masquerading process names.

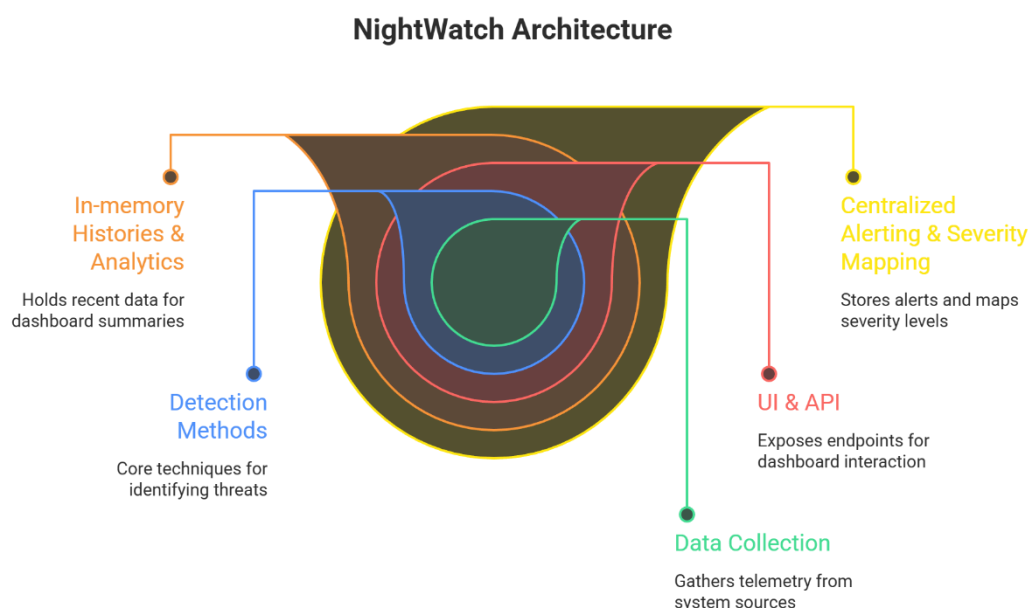
Network Monitor: Reads active network connections and records local and remote IPs and ports. Flags connections involving suspicious ports, blacklisted IP addresses or excessively high connection counts from a single process.

File System Monitor: Watches selected directories such as Downloads, Desktop, AppData and Startup folders. Detects creation and modification of executable and script files. Optionally computes file hashes and checks them against a known-bad list.

Optional Registry and Event Log Monitor: Monitors Run and RunOnce keys for new or modified entries indicating persistence. Reads selected Windows Security and System event IDs that may indicate brute-force attempts or log tampering.

Detection and Scoring Engine Assigns a numeric score to observed behaviour based on heuristic rules. Higher scores correspond to more suspicious behaviour (such as encoded PowerShell scripts or known malicious hashes). Converts numeric scores and specific flags into severity levels such as Low, Medium, High or Critical. Generates clear and concise messages explaining why an alert was raised.

Alert Management Stores alerts in a deque (bounded queue) to keep only recent alerts in memory. Each alert contains time, category, source module, severity, message and optional extra data such as PID, file path or IP address. Maintains counters of alerts by category and severity for statistics displayed on the dashboard.



Web Dashboard (Flask Application)

Implements REST API endpoints such as:

/api/summary for overall CPU, memory, alert counts and process/network counts.

/api/processes for the current list of processes and their flags or scores.

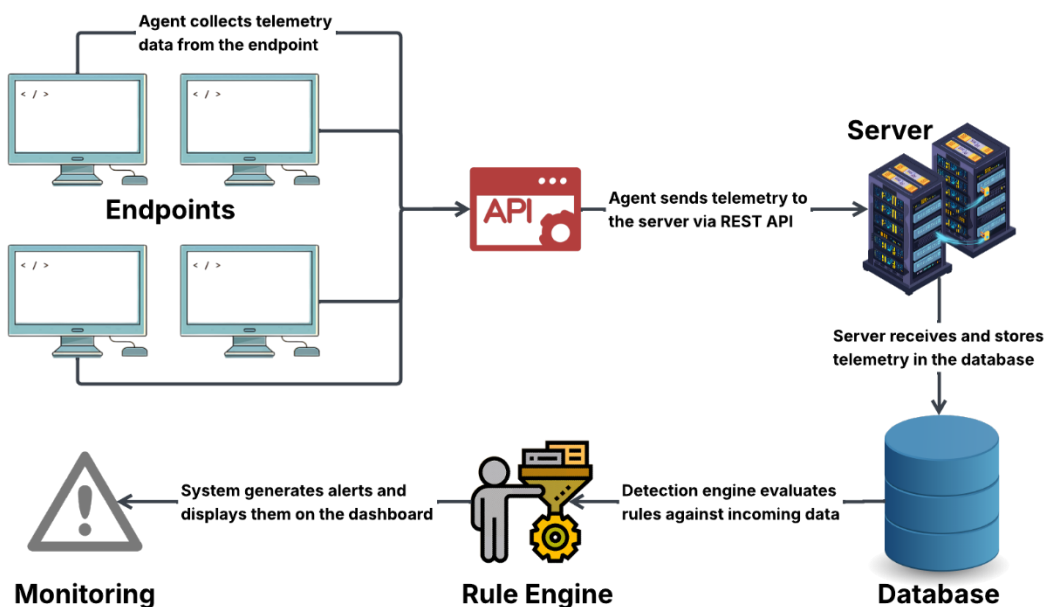
/api/network for current network connections and top remote Ips.

/api/alerts for recently generated alerts.

/api/logs and /api/stats for engine logs and resource-usage history.

Serves an HTML page that uses JavaScript to periodically call these APIs and refresh the displayed data. Presents tables for alerts, processes and network connections, and charts for CPU and memory usage.

This modular design makes it easier to implement additional features in the future, such as new detection rules, support for more event sources or integration with external systems.



CHAPTER 4

IMPLEMENTATION AND UI

Implementation is the phase where design is transformed into actual code and running components. For NightWatch, the implementation is focused on writing multi-threaded Python code that can safely collect telemetry, analyse it and expose it through a web interface. At the same time, the user interface must be simple, responsive and easy to interpret.

This chapter discusses how the monitoring threads are implemented, how data structures are shared and protected with locks and how the Flask-based dashboard is created. It also explains the layout and functionalities offered to the user.

4.1 Implementation Details

The implementation of NightWatch revolves around the monitoring engine and the Flask application. The core ideas include multi-threaded execution, safe access to shared data structures and clear separation between data collection and presentation.

Key implementation details are:

Multi-threaded Monitoring: Separate threads are created for process monitoring, network monitoring, file system monitoring, anomaly detection and statistics collection. Each thread runs an infinite loop with a sleep interval so that system resources are not overused. Global locks are used to control access to shared dictionaries and lists, preventing race conditions.

Shared Data Structures: Process information is stored in `PROCESS_DATA`, mapping process IDs to details and detection flags.

Network connections are stored in `NETWORK_DATA`, with each entry containing process association, addresses, ports and suspicion flags.

File information is stored in `FS_STATE`, mapping file paths to size and modification time.

Alerts are stored in `ALERTS`, a deque that holds recent alerts for display. Engine log messages are stored in `LOG_LINES`, another deque used to debug the behaviour of the engine.

Hashing and Reputation Checking: A helper function computes SHA-256 hashes for relevant files using hashlib. A small list or dictionary of known malicious hashes is maintained in configuration. If a file hash matches the known list, a Critical alert is generated and logged.

Configuration: Important parameters such as directory paths, suspicious ports, scan intervals and baseline windows are stored in a configuration structure. This makes it easy to tune the system for different environments without changing core logic.

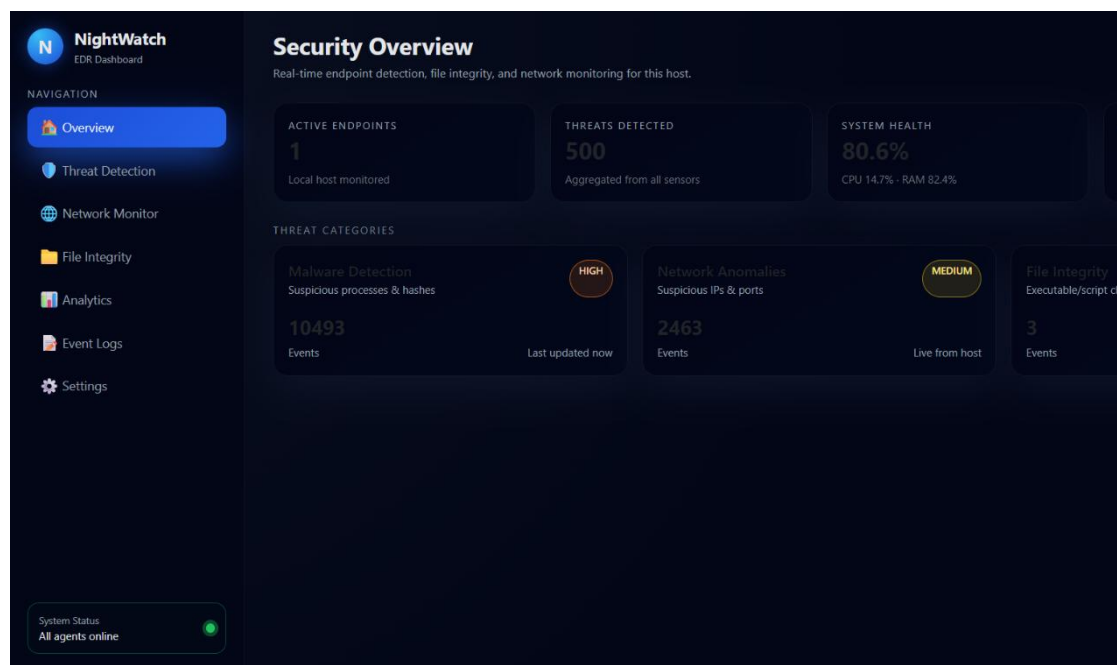
The code is written in a modular style with helper functions for common tasks, making it ideal for reading and modification by students.

4.2 User Interface (Dashboard)

The user interface is implemented as a Flask application that serves an HTML dashboard and exposes JSON APIs. From the user's point of view, the dashboard is the main way to interact with NightWatch and understand what is happening on the endpoint.

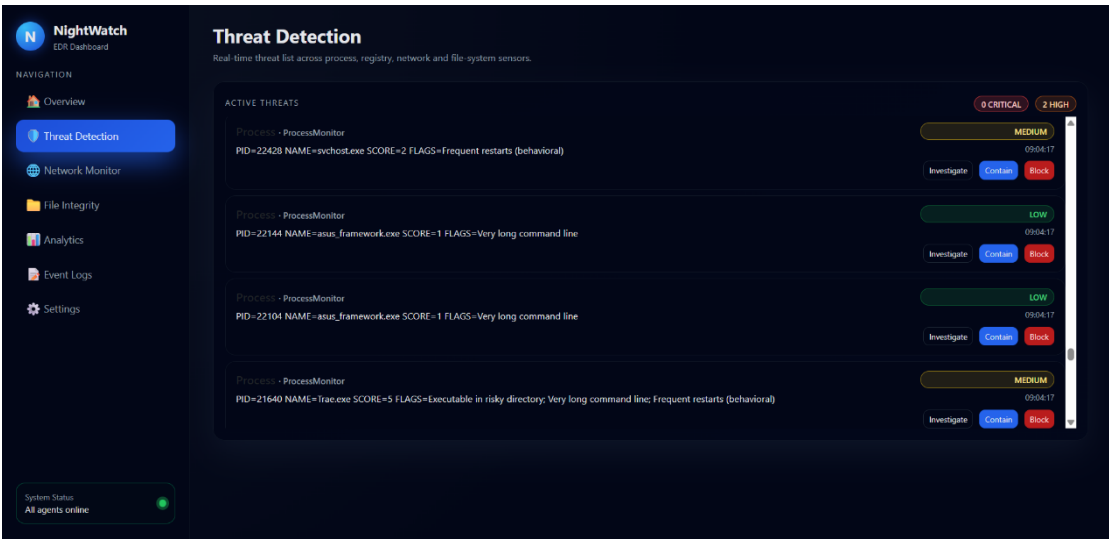
The main UI sections are:

Dashboard / Home Page: Shows a summary of CPU and memory usage. Displays the total number of alerts and a breakdown by severity and category. Gives a quick, high-level overview of the system's security state.

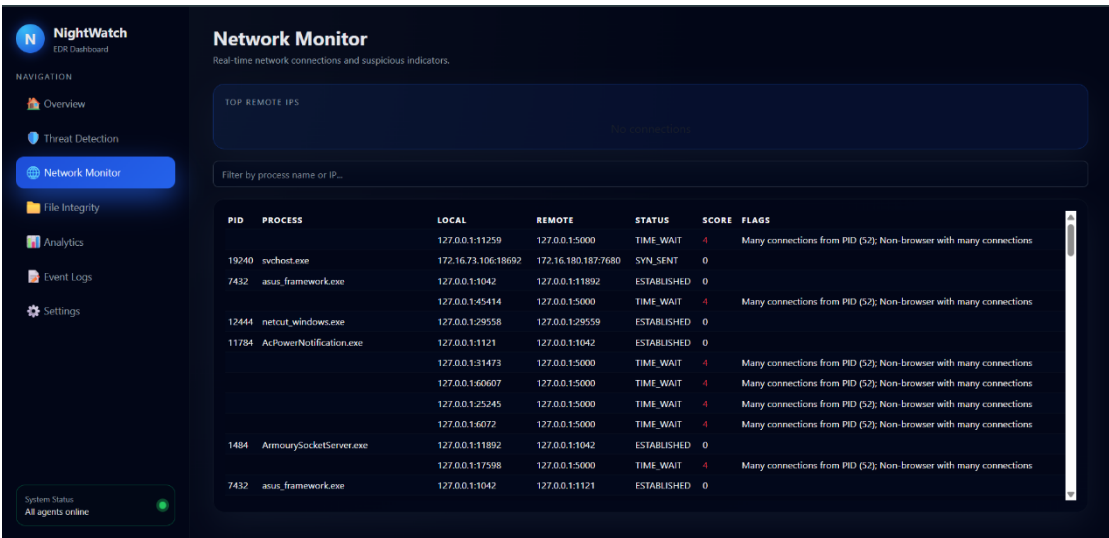


Alerts Page: Presents a table of recent alerts with columns such as time, category, severity, source and message. Uses colour coding to highlight severity levels so that critical issues can be noticed quickly. May allow simple filtering or sorting based on severity or time.

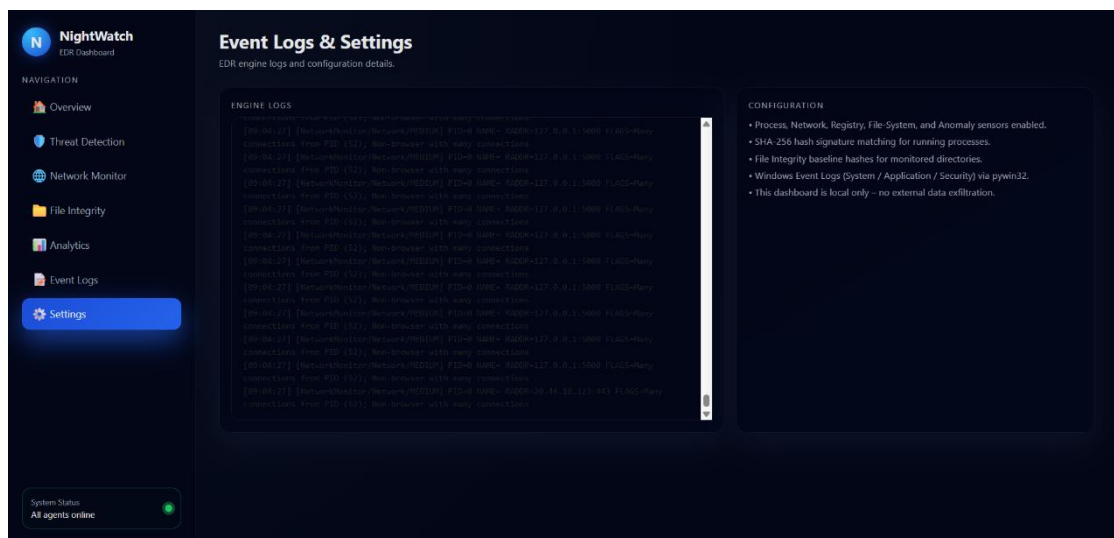
Processes Page: Lists active processes with PID, name, CPU and memory usage and any associated detection flags. Helps the user identify processes that are behaving unusually or are linked to alerts.



Network Page: Shows active network connections, including local and remote IPs and ports and the owning process. Provides a summary of frequently contacted remote IPs. Helps to identify potential C2 channels or suspicious external communication.



Logs and Statistics Page: Displays internal engine logs for debugging. Plots CPU and memory usage over time to show resource patterns.



This simple but effective dashboard enables users to observe and understand endpoint activity in real time, similar to how analysts use professional EDR consoles.

CHAPTER 5

SOFTWARE TESTING

Software testing ensures that the implemented system behaves correctly under different conditions and that it can handle both expected and unexpected inputs. For a security-oriented project like NightWatch, testing is especially important because detection logic and stability directly affect the usefulness of the tool.

This chapter describes the overall testing strategy for NightWatch, explains the methods used and lists representative test cases that were performed to validate the system.

5.1 Testing Approach

NightWatch was tested using a combination of unit testing, integration testing and scenario-based testing. The aim was to verify that individual modules work correctly, that modules cooperate as expected and that realistic attack-like scenarios trigger the appropriate alerts.

The main approaches used were:

Unit Testing: Individual helper functions such as process evaluation, hash computation and alert creation were tested with controlled input values. Boundary conditions such as extremely long command lines or missing file paths were checked.

Integration Testing: The complete flow from monitoring threads through detection logic to alert generation and dashboard display was tested. Multiple modules were run together to ensure data consistency and correct API responses.

Scenario-Based Testing: Realistic scenarios were executed on the endpoint to simulate attack behaviour, such as running encoded PowerShell commands, dropping executables in the Startup folder and using suspicious ports. Observed alerts were checked against expectations.

Robustness and Negative Testing: Tests were performed with disconnected dashboard, sudden termination of processes and invalid configurations. The goal was to confirm that the system continues running without crashes and logs errors properly.

5.2 Representative Test Cases

Some of the key test cases that were designed and executed include:

Normal System Activity Description: Use the endpoint normally with browsers, editors and development tools. **Expected Result:** Only minimal alerts are generated, and resource usage stays within normal range.

Execution from Risky Directory Description: Run an executable from the temporary or Downloads folder. **Expected Result:** An alert is generated indicating execution from a risky directory with Medium or High severity.

Encoded PowerShell Command Description: Run a PowerShell process with a base64-encoded command. **Expected Result:** A High or Critical severity alert is generated stating suspicious PowerShell activity.

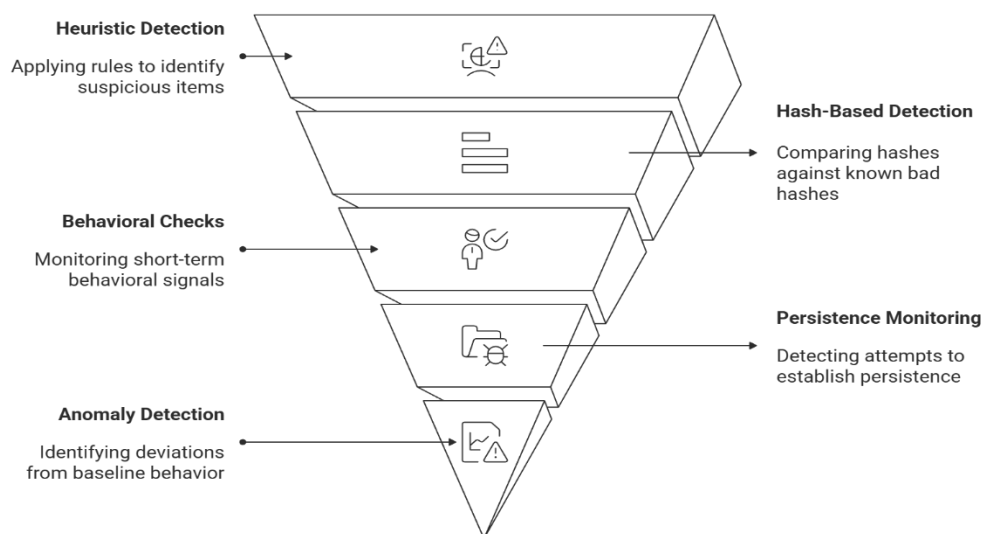
Suspicious Network Port Description: Open a connection on a non-standard port commonly used for backdoors. **Expected Result:** Network monitor creates an alert for suspicious port usage.

New File in Startup Folder Description: Copy an executable or script into the Startup folder. **Expected Result:** File system monitor detects the new file and generates a persistence-related alert.

Known Malicious Hash Description: Configure a test hash as malicious and scan a file with that hash. **Expected Result:** A Critical alert is generated, clearly mentioning the hash match.

Offline Scenario Description: Temporarily stop the dashboard or block the API while the agent is running. **Expected Result:** The monitoring engine continues functioning, and alerts are kept locally without crashing.

These test cases show that NightWatch is capable of handling benign, suspicious and error scenarios and that the detection logic functions as intended.



CHAPTER 6

CONCLUSION

Conclusion is the stage where we reflect on what has been achieved through the project and how it meets the initial objectives. For NightWatch – An Intelligent EDR Framework, the conclusion summarizes both technical outcomes and learning outcomes related to endpoint security and system design.

The project successfully demonstrates that a Python-based, educational EDR-style framework can be built using open-source tools and modest hardware. It proves that fundamental EDR concepts such as telemetry collection, detection rules, scoring and dashboard visualization can be implemented in a clear and modular manner.

Built a monitoring engine capable of tracking processes, network connections and file system activity. Implemented heuristic-based detection logic that identifies suspicious behaviours such as risky directory execution, encoded commands and repeated connections. Generated structured alerts with severity levels and descriptive messages. Created a Flask-based dashboard that visualizes alerts, processes, network data and statistics in real time. Gained practical experience with multi-threaded programming, system monitoring and web application development.

The framework achieves its primary goal of serving as a learning and experimentation platform for EDR concepts. It is not intended to replace commercial products but to help students and researchers understand how such products work internally.

Summary

The summary chapter provides a concise review of the entire project and outlines the scope for future improvement. It connects the problem statement, design choices, implementation effort and testing results into a single, coherent view.

NightWatch was conceived to fill the gap between theoretical understanding of EDR and hands-on experience. Starting from the limitations of traditional antivirus solutions and the high cost and complexity of commercial EDR products, the project defined clear objectives: to build a simple, Python-based EDR-style system that could monitor endpoints, detect suspicious behaviour and show alerts in a dashboard.

Through this project we analysed requirements and verified the feasibility of building such a system using open-source tools. Designed a modular architecture with separate monitoring, detection, alerting and user interface components. Implemented multi-

threaded monitoring for processes, network and file system, along with hashing and basic anomaly ideas. Built a Flask-based web dashboard to present real-time data to the user in an intuitive way. Tested the system with realistic scenarios and validated that the alerts and statistics matched expectations.

Future scope for NightWatch includes:

- Adding machine-learning-based anomaly detection to reduce false positives and identify new kinds of threats.
- Integrating deeper Windows event log and registry analysis for richer telemetry.
- Kernel driver based exploits, syscall hooking, api hooking and driver or kernel level malicious software detection.
- Extending support to Linux and macOS endpoints to move towards an XDR-like system.
- Implementing automated response actions such as terminating malicious processes or quarantining files.
- Storing telemetry and alerts in a database for long-term analysis and correlation.

In conclusion, NightWatch provides a strong foundation for exploring endpoint security, understanding EDR behaviour and experimenting with detection strategies. It is well suited for academic use, cybersecurity training and personal research.